

Bascule vers la production :

Plan_stockage_allium

Choix retenu

On **installe la nouvelle application sur la production**, à la même adresse et sur le même Firebase. La base de test **ne devient pas** la production.

	A. Installer sur la production (retenu)	B. Faire du test la production
Données du plan	Restent en place , aucun transfert	Export puis import : les mouvements saisis entre les deux sont perdus, sauf arrêt de l'activité
Comptes	Inchangés : mêmes e-mails, mêmes mots de passe, mêmes rôles	Tous à recréer ; mots de passe nouveaux (Firebase ne permet pas de les copier depuis la console)
Adresse	Inchangée : <code>https://fabianleg29.github.io/Plan_stockage_allium/</code>	Nouvelle adresse à communiquer à tout le monde
Données de test	Restent dans le projet de test	Mélangées aux vraies données
Retour arrière	15 min, sans perte (le format des données ne change pas)	Difficile : il faudrait refaire la bascule dans l'autre sens

Ce qui rend l'option A sûre :

- `planStockage/main` garde exactement son format.
- Le code du plan (`plan.html`) est l'ancien code, à quelques retouches près.
- Tout le reste est un **ajout** : de nouvelles collections et un champ `roleRelevés` dans `users`.

Points d'attention

1. Les utilisateurs (le point le plus sensible)

Les nouvelles règles refusent l'accès au plan à tout compte **sans fiche** `users` (`estConnu()`). L'ancienne application, elle, laissait ces comptes en Consultation.

Avant de publier les règles, il faut donc vérifier que **chaque compte** de Authentication a une fiche dans `users` :

- **Comparer les deux listes** : Authentication > Users d'un côté, Firestore > `users` de l'autre. Il faut autant de fiches que de comptes, et l'ID de chaque fiche doit être exactement l'UID du compte.
- **Vérifier le champ** `role` de chaque fiche : il doit valoir `viewer`, `editor` ou `admin`, en anglais. Une valeur comme « gestion » donne Consultation, comme dans l'ancienne application.
- **Compléter les fiches manquantes**, au moins `email` et `role: "viewer"`. C'est à faire dans la console, ou depuis l'ancien onglet Comptes **avant** la bascule.

- **Après la bascule**, l'admin attribue les rôles relevés (`roleRelevés`) dans **Administration > Comptes et accès**. Tant que ce n'est pas fait, personne n'a accès aux relevés (valeur par défaut : Aucun accès).

2. Les données

- **Aucune donnée du plan n'est copiée ni transformée** à la bascule.
- **Relevés saisis dans la base de test** : ils **ne sont pas repris**, car ce sont des données de test. S'il y en a de vrais à garder, prévoir un export Excel et JSON depuis le test **avant** la bascule (un import pourra être ajouté si besoin).
- **Archivage de l'historique du plan** : il reste sur « **Jamais** » au moins un mois, pour garder le retour arrière simple.

3. L'ordre des opérations

D'abord le code, ensuite les règles :

- Les **nouvelles règles** refusent la création d'un relevé sans auteur. L'ancien code ne crée pas de relevés, donc ce n'est pas un problème pour lui.
- Le **nouveau code** fonctionne avec les **anciennes règles** pour le plan. Les relevés et l'Administration ne marchent qu'une fois les nouvelles règles publiées.

Déroulé

Prévoir **1 h**, à un moment calme, en prévenant les utilisateurs de recharger la page après la bascule.

#	Étape	Qui	Vérification
1	Sauvegarde du plan : ancien onglet Cellules > Exporter une sauvegarde (.json). Ranger le fichier sur le NAS ou le dossier partagé, jamais dans GitHub	Admin	Fichier ouvert, il contient <code>cellules</code> , <code>lots</code> et <code>transactions</code>
2	Chiffres de contrôle : total des palox, total du Stock global, nombre de lignes de l'Histoire	Admin	Notés dans la feuille ci-dessous
3	Règles actuelles : Firestore > Règles, copier le texte dans <code>regles-avant-bascule.txt</code> (même dossier)	Admin	Fichier rangé
4	Comptes : contrôle Authentication ↔ <code>users</code> (point 1), fiches manquantes complétées, liste des rôles notée	Admin	Autant de fiches que de comptes
5	Code : session Claude Code ouverte sur le dépôt Plan_stockage_allium avec la consigne ci-dessous	Claude Code	Comparaison du code validée, étiquette <code>avant-bascule</code> posée
6	Publication : attendre le déploiement GitHub Pages (Actions > pages build and deployment ✓), puis Ctrl + F5 sur l'adresse habituelle	Admin	L'accueil s'affiche avec 3 tuiles
7	Règles : publier le <code>firestore.rules</code> du dépôt de production dans la console	Admin	Le plan s'ouvre ; les chiffres de contrôle sont identiques

#	Étape	Qui	Vérification
8	Réglages des relevés : ouvrir Administration > Relevés de température (les 15 chambres se créent), vérifier les familles, ajouter les produits du contrôle à cœur et les motifs	Admin	Liste enregistrée
9	Rôles relevés : Administration > Comptes et accès, colonne Relevés	Admin	Chaque opérateur voit la tuile Relevés
10	Essai réel : un relevé frigo, un contrôle à cœur, un export Excel, puis une entrée et une sortie de stock dans le plan	Admin et un opérateur	Tout fonctionne ; chiffres de contrôle cohérents

Consigne à donner à la session Claude Code sur Plan_stockage_allium (étape 5)

À coller tel quel dans une **nouvelle** session ouverte sur le dépôt `Plan_stockage_allium` :

Basculer de l'application vers la version à trois modules, développée et validée dans le dépôt `FabianLeg29/STOCKAGE-PLAN-TEMP-TEST` (branche `main`, lecture seule : ne rien y modifier).

- Sur la branche publiée par GitHub Pages (`claude/stockage-echalotes-oignons-6m4x91`), pose l'étiquette Git `avant-bascule` sur le dernier commit, puis pousse-la.
- Copie l'actuel `index.html` en `ancien-plan.html`, sans le modifier.
- Récupère depuis le dépôt de test : `index.html`, `plan.html`, `releves.html`, `admin.html`, `xlsx-releves.js`, `firestore.rules`, `SETUP-FIREBASE.md`, `CLAUDE.md` et `BASCULE.md`. **Ne prends pas** `prototype-plan-stockage-releves.html`.
- Dans `index.html`, `plan.html`, `releves.html` et `admin.html`, remplace la configuration Firebase du projet de test par celle de production, reprise **à l'identique** de l'ancien `index.html` (projet `plan-de-stockage-alliums`). Ensuite, vérifie qu'il ne reste **aucune** trace de `stockage-plan-temp-test` dans le dépôt.
- Adapte `CLAUDE.md` et `SETUP-FIREBASE.md` à la production : projet `plan-de-stockage-alliums`, adresse `https://fabianleg29.github.io/Plan_stockage_allium/`, branche publiée. Dans `index.html`, mets une nouvelle valeur à `VERSION`.
- Vérifications avant l'envoi :
 - `node --check` sur chaque `<script>` en ligne et sur `xlsx-releves.js` ;
 - comparaison de `plan.html` avec `ancien-plan.html` : elle ne doit montrer que le retrait des onglets Cellules et Comptes, la barre de module, le blocage du rôle `none` et, cette fois, aucune différence de configuration Firebase.
- Montre-moi la liste des fichiers et le résultat des vérifications, **puis** pousse sur la branche publiée. Ne publie pas les règles : je le fais moi-même dans la console.

Retour arrière (si besoin, environ 15 min)

- Code** : dans une session sur `Plan_stockage_allium`, demander : « Remets la branche publiée dans l'état de l'étiquette `avant-bascule`, par un revert, sans effacer l'historique Git ». L'ancien plan revient à la même adresse en quelques minutes.
- Règles** : dans la console, coller `regles-avant-bascule.txt` (ou republier la version précédente depuis l'historique des règles), puis cliquer sur Publier.
- Données** : en temps normal, **rien à restaurer**. Les mouvements faits depuis la bascule sont au bon format et l'ancien plan les affiche. Les relevés, les réglages des relevés et les champs

`roleRelevés` restent dans Firebase, ignorés par l'ancien plan. N'importer la sauvegarde de l'étape 1 que si le stock est abîmé, en notant d'abord les mouvements faits depuis la bascule.

4. **Contrôle** : comparer les chiffres de contrôle, en tenant compte des mouvements faits entre-temps.

Garder l'étiquette `avant-bascule`, `ancien-plan.html`, la sauvegarde JSON et `regles-avant-bascule.txt` **au moins un mois**.

Feuille de bascule

Point de contrôle	Avant	Après	Après un retour éventuel
Total des palox			
Total du Stock global			
Nombre de mouvements (Historique)			
Comptes Authentication = fiches <code>users</code>	☐		
Sauvegarde JSON rangée (date, emplacement)			
<code>regles-avant-bascule.txt</code> rangé	☐		
Étiquette <code>avant-bascule</code> posée	☐		
Visa			

Après la bascule : le dépôt de test

- **Projet Firebase** `stockage-plan-temp-test` : à garder, pour tester les prochaines évolutions avant de les reporter en production. Il peut être supprimé plus tard sans effet sur la production.
- **Dépôt** `STOCKAGE-PLAN-TEMP-TEST` : à garder comme terrain d'essai. GitHub Pages peut y être désactivé pour éviter toute confusion d'adresse.